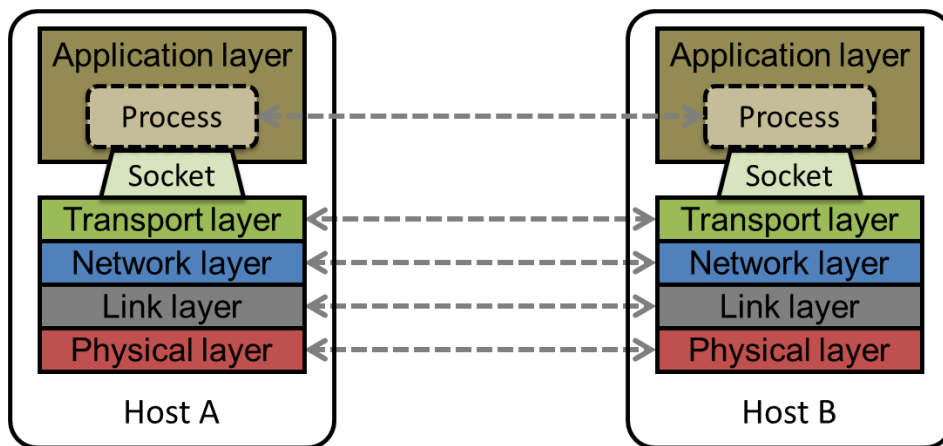


Java Socket/多线程/线程池编程讲义

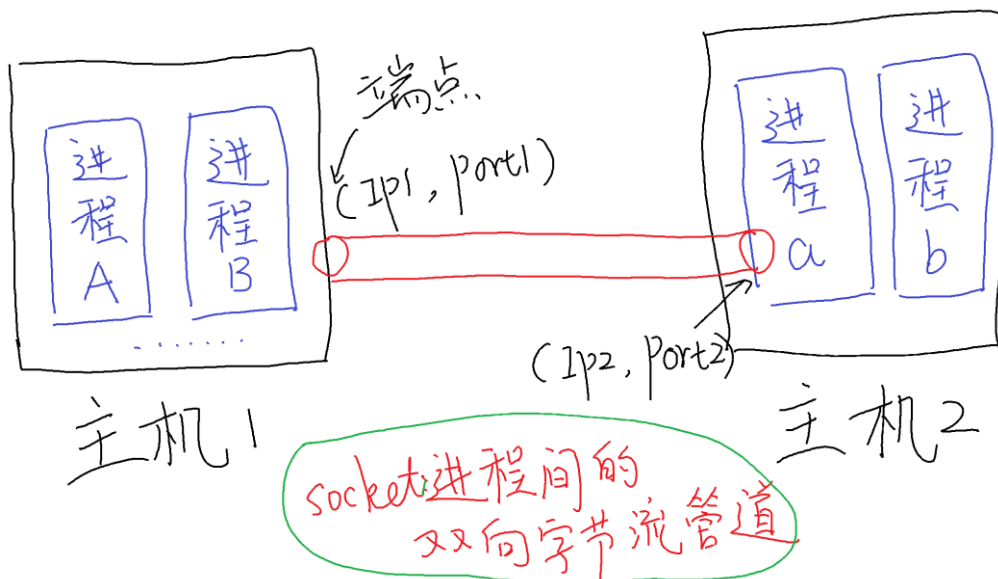
一. Socket 基本概念

1. 什么是 **Socket**: 传输层和网络层提供给应用层的标准化服务接口（或称为编程接口）。应用程序利用 Socket 接口，可以实现不同主机中不同进程之间的点对点通信。应用程序不用关心底层网络通信细节，比如链路中经过了几个路由器、有线网络还是无线网络等。应用程序只是感觉在跨主机的两个进程之间搭建了一个端到端的虚拟双向通信管道。



2. Socket 分类:

- (1) **流式 Socket**: 对 TCP 面向连接传输服务的封装。创建一个流式 Socket，相当于在跨主机的两个进程之间搭建了一个端到端的虚拟双向通信管道。



一个流式 Socket 的一个端点用一个二元组表示 (IP 地址, 端口号)。

三次握手:

滑动窗口协议:

流式 Socket 的发送函数向应用层报告某个消息发送成功之后, 该消息就一定达到了接收方的接收缓冲区。(但不保证对方的应用程序一定成功接收了该消息。)

(2) **数据报 Socket**: 对 UDP 传输服务的封装。数据报 Socket 的发送函数向应用层报告某个消息完毕之后, 该消息有可能会在传输过程中丢失。

3. 客户端与服务器端角色

(1) **服务器端**: 必须先于客户端运行, 而且要长久保持运行。被动等待客户端发起三次握手, 等待客户端的请求, 并为客户端服务。

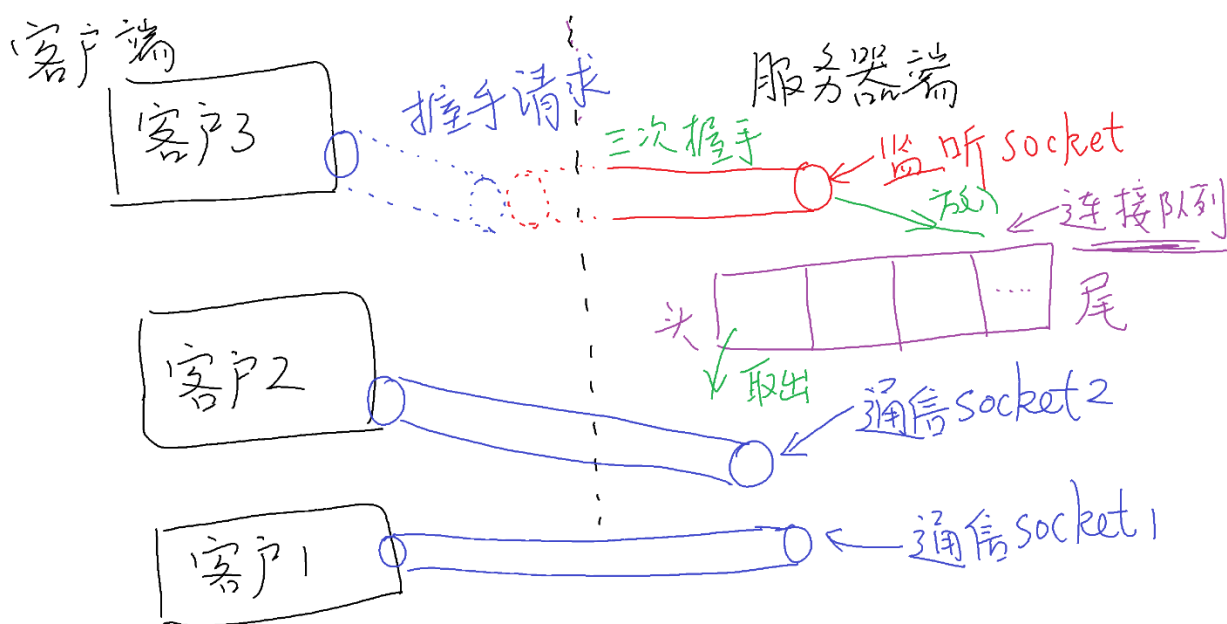
(2) **客户端**: 只在有需求时才启动。主动向服务器端发出三次握手请求, 连接建立之后发送服务请求, 并等待服务结果。

4. Socket 服务器端的几个重要概念:

(1) **监听 Socket 对象**: 负责在指定的 IP 地址和指定的端口上监听来自于不同客户端的连接请求, 并实现三次握手。握手成功之后将已经建立的 TCP 连接记录放入“连接队列”。三次握手、与客户端建立 TCP 连接、将 TCP 连接信息放入“连接队列”以及维护“连接队列”的工作都是由操作系统内核模块自动完成的。可以把这个内核模块想象成一个在后台默默工作的线程。

(2) **连接队列**: 存放 TCP 连接信息的队列。由操作系统自己维护。

(3) **通信 Socket 对象**: 操作系统内核模块从“连接队列”中每取出一个 TCP 连接记录, 就关于该 TCP 连接创建一个新的 Socket 与客户端对接到一起。该 Socket 专门负责客户端与服务器端的实际数据通信。



二. 客户端源码分析

```
1 import java.io.*;
2 import java.net.*;
3
4 public class EchoClient {
5
6     public static void main(String[] args) throws Exception {
7
8         String userInput = null;
9         String echoMessage = null;
10
11         BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in));
12
13         Socket socket = new Socket("127.0.0.1", 8189);
14         System.out.println("Connected to Server");
15
16         InputStream inStream = socket.getInputStream();
17         OutputStream outStream = socket.getOutputStream();
18         BufferedReader in = new BufferedReader(new InputStreamReader(inStream));
19         PrintWriter out = new PrintWriter(outStream);
20
21
22         while((userInput=stdIn.readLine())!=null)
23         {
24             out.println(userInput);
25             out.flush();
26             echoMessage = in.readLine();
27             System.out.println("Echo from server: " + echoMessage);
28         }
29
30         socket.close();
31     }
32 }
33 }
```

← 向服务器 127.0.0.1 发出连接请求

in对象

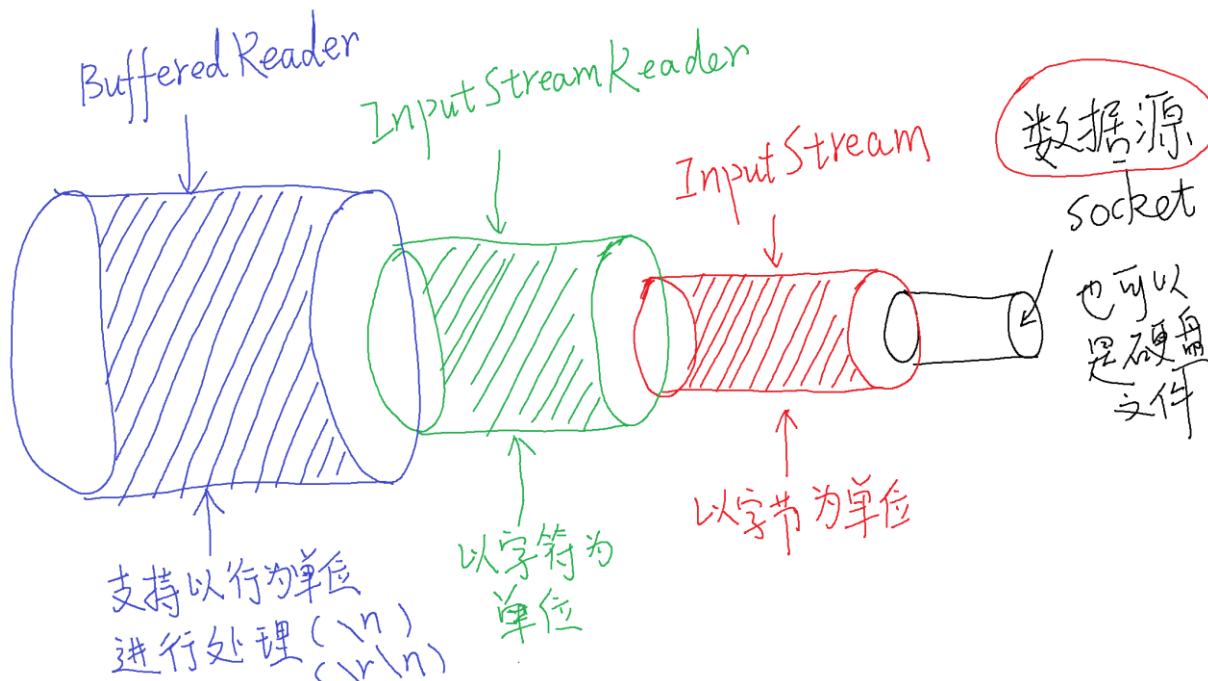
out对象

client

Server

InputStream 、 InputStreamReader 、 BufferedReader 的区别

提问：字节与字符有什么区别？



三. 服务器端源码分析

```
1 import java.io.*;
2 import java.net.*;
3
4 public class EchoServer {
5     public static void main(String[] args) throws Exception {
6         Socket clientSocket = null;
7         ServerSocket listenSocket = new ServerSocket(8189);
8
9         System.out.println("Server listening at 8189");
10        clientSocket = listenSocket.accept();
11        System.out.println("Accepted connection from client");
12
13        InputStream inStream = clientSocket.getInputStream();
14        OutputStream outStream = clientSocket.getOutputStream();
15        BufferedReader in = new BufferedReader(new InputStreamReader(inStream));
16        PrintWriter out = new PrintWriter(outStream);
17
18        String line = null;
19        while((line=in.readLine())!=null) {
20            System.out.println("Message from client:" + line);
21            out.println(line);
22            out.flush();
23        }
24        clientSocket.close();
25        listenSocket.close();
26    }
27 }
```

创建监听 socket 对象
并在 8189 端口监听
握手请求

从连接队列中取出 1
个连接记录并创建
新的
通信 socket

in: 输入流
out: 输出流

以上源码中，listenSocket 是监听 Socket，clientSocket 是通信 Socket。

四. 多线程

1. 线程的概念：

- 一个独立的执行线路。
- 不同的线程在逻辑上相互独立地并行地执行。
- 属于同一个进程的多个线程共享该进程的内存地址空间。
- 每一个线程都有自己的“主函数”。线程启动后，其主函数开始执行；线程主函数执行完毕，线程的生命周期就结束了。

2. Java 的线程对象：

- Java 线程对象是对操作系统线程对象的一个封装
- 一个 Java 线程对象一般都是 Thread 类（JDK 实现的基类）的子类。子类需要重载 run 函数，作为自己的线程主函数。
- Java 线程对象的构造函数（下面源程序中 MyThread(int t)函数）可以接收一些初始化状态或初始化数据。

3. 多线程编程举例：

```

1 class MyThread extends Thread{
2     private int ticket;
3
4     MyThread(int t) {
5         ticket = t;
6     }
7
8     public void run(){
9         for(int i=0;i<ticket;i++){
10             if(ticket>0){
11                 System.out.println(this.getName()+" sold ticket"+ ticket--);
12             }
13         }
14     }
15 };
16
17 public class ThreadTest {
18     public static void main(String[] args) {
19         MyThread t1=new MyThread(10);
20         MyThread t2=new MyThread(15);
21         MyThread t3=new MyThread(30);
22         t1.start();
23         t2.start();
24         t3.start();
25     }
26 }

```

线程主函数。

3个售票员并行售票。
不同售票员最开始时拿到的票数不同
10, 15, 30

五. 多线程实现 Socket 服务器端

```

1 import java.io.*;
2 import java.net.*;
3
4 public class MultiThreadEchoServer {
5     public static void main(String[] args) throws Exception {
6         ServerSocket listenSocket=new ServerSocket(8189);
7         Socket socket=null;
8
9         int count=0;
10        System.out.println("Server listening at 8189");
11
12        while(true){
13            socket=listenSocket.accept();
14            count++;
15            System.out.println("The total number of clients is " + count + ".");
16            ServerThread serverThread=new ServerThread(socket);
17            serverThread.start();
18        }
19    }
20 }

```

创建监听 socket, 开始监听

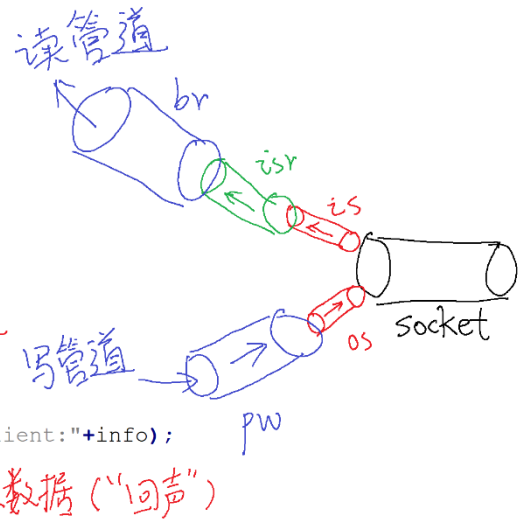
从连接队列中取出一个连接记录创建一个新的通信 socket 存放到 socket 变量中

创建一个新的线程, 以上面新创建的通信 socket 作为该线程的初始状态。

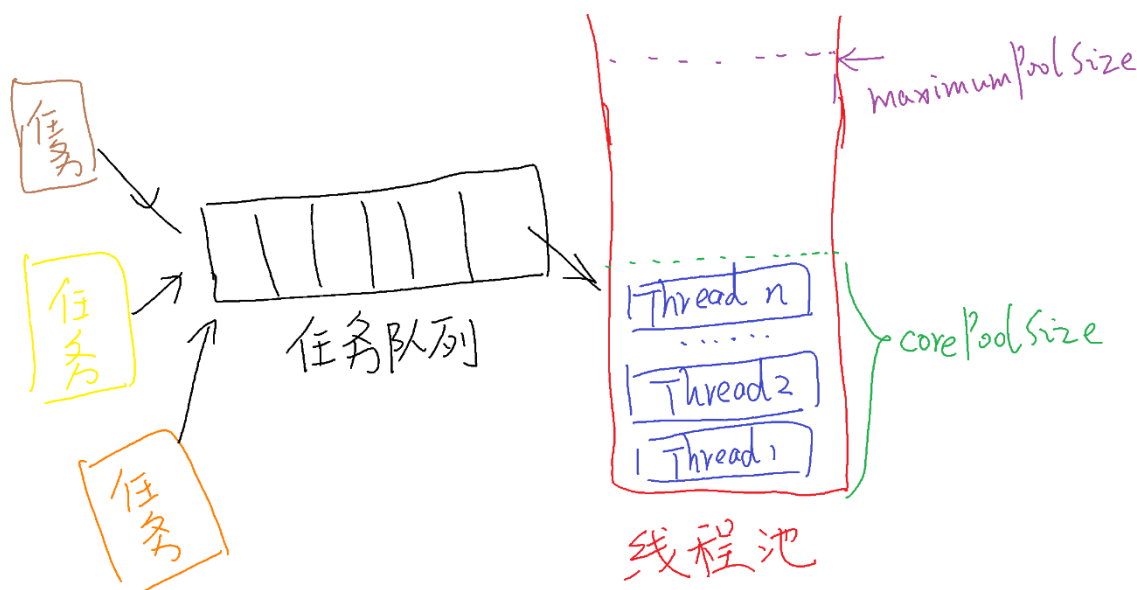
```

1 import java.io.*;
2 import java.net.*;
3
4 public class ServerThread extends Thread {
5
6     Socket socket = null;
7
8     public ServerThread(Socket socket) {
9         this.socket = socket;
10    }
11
12    public void run(){
13        InputStream is=null;
14        InputStreamReader isr=null;
15        BufferedReader br=null;
16        OutputStream os=null;
17        PrintWriter pw=null;
18        try {
19            is = socket.getInputStream();
20            isr = new InputStreamReader(is);
21            br = new BufferedReader(isr);
22            os = socket.getOutputStream();
23            pw = new PrintWriter(os);
24            String info=null;
25            while((info=br.readLine())!=null){
26                System.out.println("Message from client:"+info);
27                pw.println(info);
28                pw.flush();
29            }
30        } catch (IOException e) {
31            e.printStackTrace();
32        }finally{
33            try {
34                if(pw!=null)
35                    pw.close();
36                if(os!=null)
37                    os.close();
38                if(br!=null)
39                    br.close();
40                if(isr!=null)
41                    isr.close();
42                if(is!=null)
43                    is.close();
44                if(socket!=null)
45                    socket.close();
46            } catch (IOException e) {
47                e.printStackTrace();
48            }
49        }
50    }
51 }

```



六.线程池



ThreadPoolExecutor 参数解析

ThreadPoolExecutor 主要有以下几个参数:

```
public ThreadPoolExecutor(int corePoolSize,  
                          int maximumPoolSize,  
                          long keepAliveTime,  
                          TimeUnit unit,  
                          BlockingQueue<Runnable> workQueue,  
                          ThreadFactory threadFactory,  
                          RejectedExecutionHandler handler)
```

参数说明

(1) corePoolSize 核心线程数量

即使没有任务执行, 核心线程也会一直存活

线程数小于核心线程时, 即使有空闲线程, 线程池也会创建新线程执行任务

设置 allowCoreThreadTimeout=true 时, 核心线程会超时关闭

(2) maximumPoolSize 最大线程数

当所有核心线程都在执行任务, 且任务队列已满时, 线程池会创建新线程执行任务。

当线程数 = maxPoolSize, 且任务队列已满, 此时添加任务时会触发 RejectedExecutionHandler 进行处理。

(3) keepAliveTime TimeUnit 线程空闲时间

如果线程数>corePoolSize，且有线程空闲时间达到 keepAliveTime 时，线程会销毁，直到线程数量=corePoolSize

如果设置 allowCoreThreadTimeout=true 时，核心线程执行完任务也会销毁直到数量=0

(4) workQueue 任务队列

```
1 import java.util.concurrent.ArrayBlockingQueue;
2 import java.util.concurrent.BlockingQueue;
3 import java.util.concurrent.RejectedExecutionHandler;
4 import java.util.concurrent.ThreadPoolExecutor;
5 import java.util.concurrent.TimeUnit;
6
7 public class ThreadPoolTest {
8     public static void main(String[] args) {
9         ThreadPoolExecutor executor = new ThreadPoolExecutor(5, 10, 200, TimeUnit.
10             MILLISECONDS, new ArrayBlockingQueue<Runnable>(5));
11
12         for(int i=0; i<15; i++){
13             MyTask myTask = new MyTask(i);
14             executor.execute(myTask);
15             System.out.println("The number of threads in the ThreadPool:" + executor.
16                 getPoolSize());
17             System.out.println("The number of tasks in the Queue:" + executor.getQueue().
18                 size());
19             System.out.println("The number of tasks completed:" + executor.
20                 getCompletedTaskCount());
21         }
22         executor.shutdown();
23     }
24 }
25
26 class MyTask implements Runnable {
27     private int taskNum;
28
29     public MyTask(int num) {
30         this.taskNum = num;
31     }
32
33     @Override
34     public void run() {
35         int sum = 0;
36
37         System.out.println("Task" + taskNum + "is running!");
38
39         try {
40             for(int i=0; i<15; i++) {
41                 sum += i;
42             }
43             Thread.currentThread().sleep(4000);
44         } catch (InterruptedException e) {
45             e.printStackTrace();
46         }
47         System.out.println("Task " + taskNum + " has been done!");
48     }
49 }
```

创建线程池

创建一个新任务

新任务入队

自定义任务

任务主函数